

EvoRec：自进化智能体推荐系统

Lingyu Mu*
阿里巴巴国际数字商业集团
北京, 中国
moulingyu.mly@alibaba-inc.com

Hao Deng*
阿里巴巴国际数字商业集团
北京, 中国
denghao.deng@alibaba-inc.com

Haibo Xing
阿里巴巴国际数字商业集团
杭州, 中国
xinghaibo.xhb@alibaba-inc.com

Jinxin Hu†
阿里巴巴国际数字商业集团
北京, 中国
jinxin.hjx@alibaba-inc.com

Yu Zhang
阿里巴巴国际数字商业集团
北京, 中国
daoji@alibaba-inc.com

Xiaoyi Zeng
阿里巴巴国际数字商业集团
杭州, 中国
yuanhan@taobao.com

Abstract

现代推荐系统的优化仍然高度依赖工程师手动迭代, 这一过程速度缓慢, 并受个人专业能力限制。基于 LLM 的智能体为自动化这一循环开辟了道路, 但现有方法仅将智能体用作代码翻译器, 不会积累方法论, 而且将搜索限制在很少能引入结构性新思想的预定义空间内。我们提出 EvoRec, 一个让推荐模型与驱动其优化的方法论共同演化的多智能体框架。四个协作智能体执行双轨循环: Research Agent 和 Code Agent 在每一轮迭代模型, Skill Evolver 则定期从持久化的历史实验 Memory 中提炼可复用的方法论。在一个公开基准和一个大规模工业数据集上的实验表明, EvoRec 相比最强基线将离线指标最多提升 5.54%。线上 A/B 测试进一步带来 1.85% 的收入提升和 1.02% 的 CTR 增益, 展现出替代传统人工优化工作流的潜力。

CCS Concepts

• 信息系统 → 检索模型与排序。

Keywords

推荐系统, 智能体, 自进化

ACM Reference Format:

Lingyu Mu, Hao Deng, Haibo Xing, Jinxin Hu, Yu Zhang, and Xiaoyi Zeng. 2026. EvoRec: 自进化智能体推荐系统. In . ACM, New York, NY, USA, 5 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 引言

推荐系统是互联网平台上连接用户与信息的基础支柱 [13, 22, 24]。现代推荐系统已从协同过滤演进为复杂的多阶段流水线 [12, 27, 30], 其中每个阶段都需要大量特征工程、架构设计和超参数调优 [10, 14, 26, 32, 35]。这一过程效率低下且受个人专业能力限制, 导致优化空间的大量区域未被探索 [4, 8]。

大语言模型 (LLM) 的最新进展使具备多步规划、工具调用和代码生成能力的智能体能够像人类工程师一样自主执行复杂的

优化工作流 [9, 21, 31]。然而, 现有工作仍存在三项局限: (1) 智能体参与浅层化。大多数方法仅使用智能体生成代码, 而由人类指定优化方向, 未能充分利用智能体的推理与规划能力 [5, 16]。 (2) 经验组件静态化。一些工作引入 Skill 或 Memory 模块 [3, 17, 29], 但这些模块部署后保持冻结, 导致相同失败反复发生, 成功策略也无法复用。 (3) 演化空间受限。少数工作 [2, 23] 探索了自进化 [11, 15, 34], 但仅限于预定义空间内的超参数搜索, 无法引入新方法。

基于上述观察, 我们旨在构建满足三项性质的自进化推荐系统: (1) 开放域探索: 能够引入结构性新方法和新见解。 (2) 无人干预运行: 整条流水线由多个协作智能体自主驱动。 (3) 自我改进的经验: 系统持续从自身演化轨迹中提炼知识, 并随时间推移不断增强。

为应对这些挑战, 我们提出 EvoRec, 一个用于推荐系统的自进化框架。EvoRec 包含三个核心组件: 自进化 Model、可进化的 Skill 库, 以及持久化 Memory 存储。Memory 以结构化形式持久保存每次迭代的完整轨迹, 包括优化假设、代码修改、训练日志、评估指标和成败归因。在执行层面, EvoRec 使用四个协作智能体。Orchestrator agent 负责协调总体优化循环, 并驱动两条自进化路径:

- (1) 模型自进化由 Research Agent 和 Code Agent 驱动。Research Agent 检索外部知识 (例如 arXiv), 并将其与 Skill 和 Memory 相结合, 生成优化假设。Code Agent 将每个假设转换为代码修改并提交训练。
- (2) Skill 自进化由 Skill Evolver 负责; 它每隔 N 轮从 Memory 中提炼可复用的方法论, 并更新 Skill 库。

模型在每一轮中依次经历假设—实现—验证循环, 而 Skill 则在积累足够经验后周期性演化。在 Orchestrator 的协调下, 两条路径通过持久化 Memory 耦合, 形成一个自我强化循环, 使模型不断增强, 方法论也日益完善。在一个公开基准和一个工业数据集上的实验表明, 相较于最强基线, 离线指标最多提升 5.54%; 线上 A/B 测试则证实收入提升 1.85%, CTR 提升 1.02%。

我们的贡献总结如下:

- 我们提出 EvoRec, 这是首个面向推荐系统的多智能体自进化优化框架, 能够在无人干预的情况下持续实现模型自进化。
- 我们设计了一种由四个协作智能体驱动的双轨自进化机制, 通过持久化 Memory 实现 Model 与 Skill 的共同演化。
- 我们在一个公开基准和一个工业广告平台上验证了 EvoRec 的有效性, 并通过线上 A/B 测试进一步展示其实际部署价值。

*对本研究贡献相同。

†通讯作者。

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

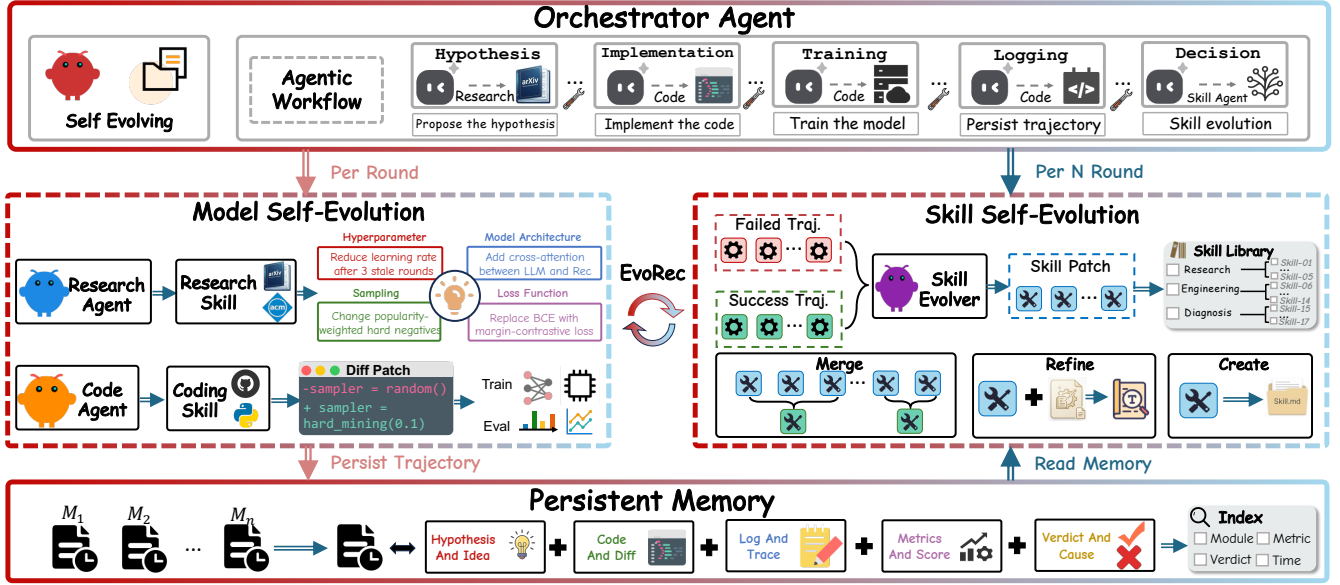


Figure 1: EvoRec 概览。四个协作智能体驱动双轨自进化：Research Agent 和 Code Agent 执行逐轮模型自进化，Skill Evolver 则定期从持久化 Memory 中提炼方法论，以更新 Skill 库。

2 方法

2.1 概述

如图 1 所示，EvoRec 由三个核心组件构成：Model、可演化的 Skill 库和结构化 Memory，并由四个协作智能体进行协调。每次迭代依次完成假设生成、代码实现、训练和经验持久化，从而形成模型与方法论协同进化的双轨循环。

2.2 Orchestrator Agent

Orchestrator 维护全局状态并协调其他三个智能体。每次迭代时，它激活 Research Agent 和 Code Agent，完成一轮模型优化，然后将完整轨迹写入 Memory。每隔 N 轮，它触发 Skill Evolver 更新 Skill 库。当达到 $T_{\max}$ 次迭代、连续 k 轮无改进或达到目标指标阈值时，循环终止。失败轮次不会被丢弃，而是作为负面样例保存在 Memory 中。

2.3 模型自进化

模型自进化由 Research Agent 与 Code Agent 联合完成，是每次 EvoRec 迭代的核心。设第 t 轮的全局状态为 $\langle \theta_{t-1}, \mathcal{S}_t, \mathcal{M}_t \rangle$ ，分别表示当前模型参数、Skill 库和 Memory 经验存储。

2.3.1 Research Agent. Research Agent 负责生成当前轮次的优化假设。它从外部知识源 $\mathcal{K}_{\text{ext}}$ （例如 arXiv、技术博客）检索候选研究方向，并结合 $\mathcal{S}_t$ 中的 Research 类方法论以及 $\mathcal{M}_t$ 中历史成功与失败案例进行权衡，输出：

$$h_t \sim \pi_R(\cdot | \mathcal{K}_{\text{ext}}, \mathcal{S}_t, \mathcal{M}_t), \quad (1)$$

其中， $h_t = \langle \text{motive}, \text{target_module}, \text{expected_gain} \rangle$ 包含优化动机、待修改的目标模块和预期收益。生成假设前，Research Agent 会执行多样性约束检查，避免连续过多轮次聚焦同一模块而陷入局部最优。通过检索 $\mathcal{M}_t$ ，它可以避开此前失败的方向；注入 $\mathcal{S}_t$ 则会使其偏向已经证明有效的探索模式。

2.3.2 Code Agent. Code Agent 将假设 h_t 转换为可执行实验。它定位相关代码，调用 $\mathcal{S}_t$ 中的 Engineering 类方法论生成最小差异修改，并提交训练任务。任务完成后，它输出包含训练日志和评估指标的轮次结果 r_t 。如果在实现期间判定 h_t 不可行，则返回拒绝信号，由 Research Agent 生成替代假设。失败运行会标注失败原因并保留，供后续 Skill 演化使用。

2.4 Memory：持久化经验存储

Memory 是 EvoRec 的经验中枢。它以结构化形式持久保存每次迭代的完整轨迹，既作为 Research Agent 的检索来源，也作为 Skill Evolver 的提炼材料。每次迭代后，Orchestrator 将该轮输出打包为一条 Memory 记录：

$$m_t = \langle h_t, \Delta c_t, \log_t, \text{metric}_t, \text{verdict}_t \rangle, \quad (2)$$

其中， h_t 是优化假设， Δc_t 是代码修改， $\log_t$ 是训练日志摘要， metric_t 是离线评估指标向量， $\text{verdict}_t \in \{\text{success}, \text{fail}, \text{neutral}\}$ 是归因标签。所有记录按时间顺序追加，形成 $\mathcal{M}_t = \{m_1, m_2, \dots, m_t\}$ 。

为了支持上游智能体高效访问，Memory 在每次写入时构建结构化索引。具体而言，系统从每个 m_t 中提取元数据字段，包括 `target_module`（修改的模型模块）、`verdict`（归因标签）、`metric_delta`（相对上一版本的指标变化）和 `timestamp`。这些字段作为倒排索引存储，支持精确的字段级筛选。Memory 本身不参与自进化，仅作为被动存储。

2.5 Skill 自进化

不同于 Memory 中存储的逐轮实验轨迹，Skill 是通过跨轮次归纳压缩生成的稳健操作原则，而非局部经验。它们以声明式自然语言表达，不绑定任何特定模型架构，可以在不同优化目标间复用。形式化地，一个 Skill 被定义为四元组 $s = \langle \text{name}, \text{scope}, \text{content}, \text{evidence} \rangle$ ，其中 `name` 是 Skill 标识符，`scope` 指定适用条件（例如“attention selection”或“loss not decreasing”），`content` 给出具体操作指南，

其粒度可从一条经验法则到多步条件流程,并随着演化不断丰富; *evidence* 指向支撑该规则的 Memory 记录,以保证可追溯性。以下是一个简短示例:

Skill #12: Hard Negative Mining

适用范围: 召回率进入平台期时的采样策略

内容: 当 $R@10$ 连续 ≥ 3 轮停滞时, 将随机负样本替换为按流行度加权的困难负样本 ($\tau \in [0.05, 0.15]$)。从 $\tau=0.1$ 开始, 并根据训练损失方差进行调整。

证据: Memory 记录 m_{14}, m_{18}, m_{22}

按功能阶段划分, Skill 分为三类: Research Skill (指导假设形成)、Engineering Skill (指导代码实现) 和 Diagnosis Skill (指导故障修复, 主要从 *verdict=fail* 案例中提炼)。Orchestrator 将每个 Skill 的 *scope* 与当前目标进行匹配, 并仅将相关条目注入智能体上下文。所有类别共享同一演化流水线, 共同构成 $\mathcal{S}_t$ 。

2.5.1 触发与批次选择。 Skill 自进化由 Skill Evolver 智能体执行, 并由 Orchestrator 每隔 N 轮 Model 迭代触发一次。在触发轮次 t , Skill Evolver 从 Memory 中选择最近 N 轮实验记录作为提炼批次:

$$\mathcal{B}_t = \{m_\tau \in \mathcal{M}_t \mid t - N + 1 \leq \tau \leq t\}, \quad (3)$$

并按 *verdict* 标签进行均衡筛选, 使成功、失败和中性案例按照预设比例 ρ 出现在批次中。这种平衡既确保提炼过程从成功案例中抽取正向模式, 也能归纳失败教训。

2.5.2 提炼与更新。 给定批次 $\mathcal{B}_t$ 和当前 Skill 库 $\mathcal{S}_t$, Skill Evolver 通过策略 π_E 生成一组候选 Skill 补丁:

$$\mathcal{C}_t = \{c_1, \dots, c_K\} \sim \pi_E(\cdot \mid \mathcal{B}_t, \mathcal{S}_t). \quad (4)$$

每个候选项 c_k 都遵循相同的四元组结构。对于每个候选项, 系统计算它与现有 Skill 库 $\sigma_k = \max_{s \in \mathcal{S}_t} \text{sim}(c_k, s)$ 的最大相似度, 并根据阈值 τ_c 应用三种更新算子之一:

- 合并: 首先将在批次内两两相似度高于 τ_c 的候选项整合为一个候选项, 避免同时引入冗余 Skill。
- 细化: 当 $\sigma_k \geq \tau_c$ 时, 将 c_k 与最相似的现有 Skill 合并并修订, 对应于方法论细化。
- 创建: 当 $\sigma_k < \tau_c$ 时, 将候选项 c_k 作为新的 Skill 加入库中, 对应于方法论扩展。

Skill 库随后更新为:

$$\mathcal{S}_{t+1} = (\mathcal{S}_t \setminus \mathcal{S}_{\text{refined}}) \cup \mathcal{S}'_{\text{refined}} \cup \mathcal{S}_{\text{new}}. \quad (5)$$

3 实验

3.1 实验设置

3.1.1 数据集与指标。 我们在一个公开数据集 (Amazon Books [6], 筛选至少有 ≥ 5 次交互的用户) 以及一个工业数据集上进行评估。工业数据集来自某东南亚电商平台 2026 年 1 月至 5 月的内部交互日志, 包含约 2.95B 次用户-物品交互、14.7M 名用户和 20.1M 个物品。我们采用留一法评估, 并报告 Recall@5/10 ($R@5/10$) 和 NDCG@5/10 ($N@5/10$) [27]。

3.1.2 基线与基础模型。 我们与基于 ID 的推荐器 (SASRec [8], S³Rec [36], PinnerFormer [18])、生成式推荐器 (HSTU [33], TIGER [20], Cobra [30]) 以及自动优化方法 (AutoML [7], 分别应用于各基础模型, 得到 Auto-MGUI 和 Auto-REG) 进行比较。我们选择 MGUI [25] (基于 ID) 和 REG4Rec [28] (生成式) 作为 EvoRec 的基础模型, 得到 Evo-MGUI 和 Evo-REG4Rec。

3.1.3 实现细节。 所有模型均使用 PyTorch 在 4 块 NVIDIA A100 GPU 上训练 [19]。四个智能体共享 Claude Opus 4.6 作为推理主干, 并设置 $N=5$ 、 $T_{\text{max}}=50$ 、 $k=10$ 、 $\tau_c=0.6$ 和 $\rho=(0.5, 0.3, 0.2)$ 。两个基础模型的训练超参数均遵循其原始论文。使用 Qwen3.7-MAX [1] 复现该流程可得到一致结论, 证实了对 LLM 主干选择的稳健性。

3.2 主要结果

表 1 报告了公开数据集和工业数据集上的结果。EvoRec-MGUI 和 EvoRec-REG4Rec 在所有数据集和指标上均取得最佳性能, 相比最强基线的相对提升最高达 5.54%。在基于 ID 和生成式两种范式上均获得一致增益, 证实了该框架的通用性。工业数据集上的提升通常大于 Books, 这是因为更丰富的特征空间提供了更广阔的优化景观。与此同时, 生成式范式中的增益略小, 因为生成模型已经融入更强的表示先验。与受限于固定配置空间的 AutoML 相比, EvoRec 始终具有优势, 因为其 Skill 自进化机制会在迭代中积累并完善优化方法, 持续发现超出预定义搜索范围的更优方向。

3.3 消融研究

为验证各核心组件的贡献, 我们以 EvoRec-MGUI 为基础模型, 在工业数据集上进行三项消融实验, 结果见表 2。

- 不使用 Skill 进化 (-2.10% $R@10$): Skill 库保持冻结。性能下降证实, 持续完善的方法对于指导后期优化至关重要。
- 不使用 Memory (-3.50% $R@10$): 缺少历史记录时, 系统会反复探索失败方向, 这证实了 Memory 在加速收敛方面的作用。
- 不使用外部知识 (-1.58% $R@10$): 仅依赖内部经验会使系统局限于已知模式, 表明外部知识能够引入结构上新颖的解决方案。

3.4 案例研究

我们使用 EvoRec-MGUI 在工业数据集上的优化轨迹, 说明 EvoRec 在 50 次迭代中的典型工作行为。表 3 列出了指标变化最显著的轮次, 其中包括成功和失败的尝试。

该轨迹涵盖超参数、采样、架构和损失设计等多种类别。并非所有尝试都成功: 第 11 轮和第 36 轮导致性能退化, 但它们作为负面样例保留在 Memory 中, 帮助 Research Agent 在后续避免类似方向。积累足够经验后, 由 Skill 驱动的轮次开始出现 (第 27、33 轮), 且均取得成功; 其中第 27 轮通过由先前采样实验提炼出的 Skill #12, 实现了最大的单轮增益 ($+2.31\%$)。这表明, 提炼后的方法比开放式探索能够提供更可靠的方向, 而两种来源发挥着互补作用。

3.5 在线 A/B 测试

我们将 EvoRec-MGUI 部署在一家东南亚领先电商公司的广告推荐平台上, 进行为期 7 天的在线 A/B 测试。对照组使用生产模型 MGUI, 实验组则将其替换为经 EvoRec 优化的变体。每组均包含均匀随机采样的 20% 用户。EvoRec-MGUI 使广告收入提升 **1.85%**, CTR 提升 **1.02%**, 二者在双侧检验下均具有统计显著性 ($p < 0.05$)。这些结果证实, EvoRec 的离线增益能够转化为真实的商业价值。

4 结论

本文提出 EvoRec, 一种用于推荐系统的多智能体自进化框架, 以自主智能体协作取代人工优化。我们指出以往基于智能体的

Table 1: Books 和 Industrial 数据集上的性能比较。“**Imp.**”表示相对于各组最强基线的相对提升(%)。每组最佳结果以**粗体**表示，次优结果以下**划线**表示。

		基于 ID 的推荐							生成式推荐						
		SASRec	S ³ Rec	PFormer	MGUI	Auto-MGUI	Evo-MGUI	提升	HSTU	TIGER	Cobra	REG4Rec	Auto-REG	Evo-REG4Rec	提升
Books	R@5	0.0472	0.0471	0.0492	0.0503	<u>0.0506</u>	0.0527	4.15%	0.0536	0.0562	0.0584	0.0587	<u>0.0592</u>	0.0608	2.70%
	N@5	0.0267	0.0264	0.0281	<u>0.0284</u>	0.0279	0.0298	4.93%	0.0307	0.0324	<u>0.0335</u>	0.0331	0.0327	0.0349	4.18%
	R@10	0.0599	0.0597	0.0623	<u>0.0635</u>	0.0630	0.0664	4.57%	0.0697	0.0733	0.0761	<u>0.0768</u>	0.0763	0.0792	3.13%
	N@10	0.0343	0.0339	0.0357	<u>0.0361</u>	0.0358	0.0381	5.54%	0.0392	0.0415	0.0429	<u>0.0425</u>	<u>0.0431</u>	0.0446	3.48%
Industry	R@5	0.0701	0.0706	0.0734	0.0741	<u>0.0745</u>	0.0775	4.03%	0.0832	0.0904	0.0965	<u>0.1016</u>	0.1008	0.1053	3.64%
	N@5	0.0919	0.0939	<u>0.0984</u>	0.0982	0.0976	0.1037	5.39%	0.1062	0.1165	0.1236	0.1294	0.1302	0.1331	2.23%
	R@10	0.0518	0.0523	<u>0.0544</u>	<u>0.0547</u>	0.0542	0.0571	4.39%	0.0613	0.0672	0.0701	<u>0.0736</u>	0.0731	0.0761	3.40%
	N@10	0.0681	0.0697	0.0741	0.0736	<u>0.0743</u>	0.0776	4.44%	0.0811	0.0871	0.0916	0.0957	<u>0.0961</u>	0.0993	3.33%

Table 2: 以 EvoRec-MGUI 为基础在工业数据集上进行的消融研究。△ 表示相对于完整模型的 R@10 相对下降。

方法	R@5	N@5	R@10	N@10	ΔR@10
EvoRec-MGUI	0.0775	0.1037	0.0571	0.0776	–
不使用 Skill 进化	0.0759	0.1015	0.0559	0.0761	-2.10%
不使用 Memory	0.0749	0.1003	0.0551	0.0750	-3.50%
不使用外部知识	0.0763	0.1021	0.0562	0.0764	-1.58%

Table 3: EvoRec-MGUI 在工业数据集上的优化过程中，R@10 变化最显著的轮次。

轮次	类别	修改	ΔR@10	来源
3	超参数	将 lr 降至 5e-4	+0.41%	Research
11	损失	Focal loss 变体	-0.82%	Research
14	采样	负样本 4→16	+0.52%	Research
19	架构	多兴趣池化 (4 个头)	+1.03%	Research
24	损失	用 InfoNCE 替换 BCE	+1.18%	Research
27	采样	困难负样本挖掘 (τ=0.1)	+2.31%	Skill #12
33	超参数	逐层 lr 衰减	+0.63%	Skill #16
36	架构	注意力头剪枝	-0.87%	Research

方法存在三个局限：参与程度浅，仅限于代码翻译；经验静态不更新；进化受限于预定义搜索空间。EvoRec 通过双轨机制解决这些问题，其中模型与 Skill 库通过持久化 Memory 共同进化。大量离线和在线实验验证了 EvoRec 在公开和工业环境中的有效性。未来工作包括多阶段级联优化，以及不同推荐场景之间的跨任务 Skill 迁移。

References

[1] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).

[2] Yin Cheng, Liao Zhou, Xiyu Liang, Dihao Luo, Tawei Lee, Kailun Zheng, Weiwei Zhang, Mingchen Cai, Jian Dong, and Andy Zhang. 2026. Let the Agent Steer: Closed-Loop Ranking Optimization via Influence Exchange. *arXiv preprint arXiv:2603.27765* (2026).

[3] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413* (2025).

[4] Hao Deng, Haibo Xing, Kanefumi Matsuyama, Moyu Zhang, Jinxin Hu, Hong Wen, Yu Zhang, Xiaoyi Zeng, and Jing Zhang. 2025. CSMF: Cascaded Selective Mask Fine-Tuning for Multi-Objective Embedding-Based Retrieval. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2122–2131.

[5] Xuegang Hao, Ming Zhang, Alex Li, Xiangyu Qian, Zhi Ma, Yanlong Zang, Shijie Yang, Zhongxuan Han, Xiaolong Han, Jinguang Liu, et al. 2025. OxygenREC: An Instruction-Following Generative Framework for E-commerce Recommendation. *arXiv preprint arXiv:2512.22386* (2025).

[6] Ruining He and Julian McAuley. 2016. Ups and Downs: Modeling the Visual Evolution of Fashion Trends with One-Class Collaborative Filtering. In *Proceedings of the 25th International Conference on World Wide Web* (Montréal, Québec, Canada) (WWW '16). International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 507–517. doi:10.1145/2872427.2883037

[7] Xin He, Kaiyong Zhao, and Xiaowen Chu. 2021. AutoML: A survey of the state-of-the-art. *Knowledge-based systems* 212 (2021), 106622.

[8] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.

[9] Fei Liu, Xinyu Lin, Hanchao Yu, Mingyuan Wu, Jianyu Wang, Qiang Zhang, Zhuokai Zhao, Yinglong Xia, Yao Zhang, Weiwei Li, et al. 2025. Recoworld: Building simulated environments for agentic recommender systems. *arXiv preprint arXiv:2509.10397* (2025).

[10] Qijiong Liu, Jieming Zhu, Quanyu Dai, and Xiao-Ming Wu. 2022. Boosting deep CTR prediction with a plug-and-play pre-trainer for news recommendation. In *Proceedings of the 29th International Conference on Computational Linguistics*. 2823–2833.

[11] Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. 2026. Skillclaw: Let skills evolve collectively with agentic evolver. *arXiv preprint arXiv:2604.08377* (2026).

[12] Lingyu Mu, Hao Deng, Haibo Xing, Jinxin Hu, Yu Zhang, Xiaoyi Zeng, and Jing Zhang. 2026Reg4rec: Reasoning-enhanced generative model for large-scale recommendation systems. Masked Diffusion Generative Recommendation. *arXiv preprint arXiv:2601.19501* (2026Reg4rec: Reasoning-enhanced generative model for large-scale recommendation systems).

[13] Lingyu Mu, Zhengxiao Liu, Zhitong Zhu, and Zheng Lin. 2025. Trust-GRS: A Trustworthy Training Framework for Graph Neural Network Based Recommender Systems Against Shilling Attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 12408–12416.

[14] Aashiq Muhamed, Iman Keivanloo, Suhan Perera, James Mracek, Yi Xu, Qingjun Cui, Santosh Rajagopalan, Belinda Zeng, and Trishul Chilimbi. 2021. CTR-BERT: Cost-effective knowledge distillation for billion-parameter teacher models. In *NeurIPS Efficient Natural Language and Speech Processing Workshop*.

[15] Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. 2026. Trace2skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158* (2026).

[16] Kesha Ou, Chenghao Wu, Xiaolei Wang, Bowen Zheng, Wayne Xin Zhao, Weitao Li, Long Zhang, Sheng Chen, and Ji-Rong Wen. 2026. Deep Research for Recommender Systems. *arXiv preprint arXiv:2603.07605* (2026).

[17] Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. 2023. MemGPT: towards LLMs as operating systems. (2023).

[18] Nikil Pancha, Andrew Zhai, Jure Leskovec, and Charles Rosenberg. 2022. Pinnerformer: Sequence modeling for user representation at pinterest. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 3702–3712.

[19] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).

[20] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al.

2023. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems* 36 (2023), 10299–10315.
- [21] Jiakai Tang, Yujie Luo, Xunke Xi, Fei Sun, Xueyang Feng, Sunhao Dai, Chao Yi, Dian Chen, Zhujin Gao, Yang Li, et al. 2025. Interactive Recommendation Agent with Active User Commands. *arXiv preprint arXiv:2509.21317* (2025).
- [22] Hanbing Wang, Xiaorui Liu, Wenqi Fan, Xiangyu Zhao, Venkataramana Kini, Devendra Yadav, Fei Wang, Zhen Wen, Jiliang Tang, and Hui Liu. 2024. Rethinking large language model architectures for sequential recommendations. *arXiv preprint arXiv:2402.09543* (2024).
- [23] Haochen Wang, Yi Wu, Daryl Chang, Li Wei, and Lukasz Heldt. 2026. Self-evolving recommendation system: End-to-end autonomous model optimization with LLM agents. *arXiv preprint arXiv:2602.10226* (2026).
- [24] Shoujin Wang, Longbing Cao, Yan Wang, Quan Z Sheng, Mehmet A Orgun, and Defu Lian. 2021. A survey on session-based recommender systems. *ACM Computing Surveys (CSUR)* 54, 7 (2021), 1–38.
- [25] Bin Wu, Xiaowen Yin, Xun Su, and Mingliang Xu. 2026. Modeling Multi-Grained User Interests for Sequential Recommendation. *IEEE Transactions on Computational Social Systems* (2026).
- [26] Chuhan Wu, Fangzhao Wu, Tao Qi, and Yongfeng Huang. 2021. Empowering news recommendation with pre-trained language models. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 1652–1656.
- [27] Haibo Xing, Hao Deng, Yucheng Mao, Jinxin Hu, Yi Xu, Hao Zhang, Jiahao Wang, Shizhun Wang, Yu Zhang, Xiaoyi Zeng, et al. 2025. Reg4rec: Reasoning-enhanced generative model for large-scale recommendation systems. *arXiv preprint arXiv:2508.15308* (2025).
- [28] Haibo Xing, Hao Deng, Yucheng Mao, Jinxin Hu, Yi Xu, Hao Zhang, Jiahao Wang, Shizhun Wang, Yu Zhang, Xiaoyi Zeng, et al. 2025. Reg4rec: Reasoning-enhanced generative model for large-scale recommendation systems. *arXiv preprint arXiv:2508.15308* (2025).
- [29] Renjun Xu and Yang Yan. 2026. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430* (2026).
- [30] Yuhao Yang, Zhi Ji, Zhaopeng Li, Yi Li, Zhonglin Mo, Yue Ding, Kai Chen, Zijian Zhang, Jie Li, Shuanglong Li, et al. 2025. Sparse meets dense: Unified generative recommendations with cascaded sparse-dense representations. *arXiv preprint arXiv:2503.02453* (2025).
- [31] Chao Yi, Dian Chen, Gaoyang Guo, Jiakai Tang, Jian Wu, Jing Yu, Mao Zhang, Wen Chen, Wenjun Yang, Yujie Luo, et al. 2025. RecGPT-V2 Technical Report. *arXiv preprint arXiv:2512.14503* (2025).
- [32] Zheng Yuan, Fajie Yuan, Yu Song, Youhua Li, Junchen Fu, Fei Yang, Yunzhu Pan, and Yongxin Ni. 2023. Where to go next for recommender systems? id-vs. modality-based recommender models revisited. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2639–2649.
- [33] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152* (2024).
- [34] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, et al. 2026. Coevoskills: Self-evolving agent skills via co-evolutionary verification. *arXiv preprint arXiv:2604.01687* (2026).
- [35] Song Zhang, Nan Zheng, and Danli Wang. 2022. GBERT: Pre-training user representations for ephemeral group recommendation. In *Proceedings of the 31st ACM international conference on information & knowledge management*. 2631–2639.
- [36] Kun Zhou, Hui Wang, Wayne Xin Zhao, Yutao Zhu, Sirui Wang, Fuzheng Zhang, Zhongyuan Wang, and Ji-Rong Wen. 2020. S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization. In *Proceedings of the 29th ACM international conference on information & knowledge management*. 1893–1902.